

UNIT-III: Software Testing & Evolution

Chapter 8 – Software Testing

Topics Covered:

- Development testing
- Test-driven development
- Release testing
- User testing

Program Testing

Program testing is an essential activity in software development aimed at ensuring a program functions as intended and uncovering defects before the software is deployed for use.

- When testing software, a program is executed using artificial data.
- The results of these test runs are then checked for errors, anomalies, or information regarding the program's non-functional attributes.
- It is crucial to understand that testing can **reveal the presence of errors, but not their absence**. This means testing can show a defect exists, but it cannot definitively prove that no other defects remain.
- Testing forms a part of a broader verification and validation (V&V) process, which also includes static validation.

Program Testing Goals

There are two primary goals for program testing:

1. **To demonstrate to the developer and the customer that the software meets its requirements.**
 - For custom software, this implies that there should be at least one test case corresponding to every requirement listed in the requirements document.
 - For generic software products, it means designing tests for all system features, including combinations of these features, that are intended for the product release.
2. **To discover situations in which the behavior of the software is incorrect, undesirable, or does not conform to its specification.**

Validation and Defect Testing

The two goals of program testing lead to two distinct types of testing:

- **Validation Testing:**
 - This type of testing aligns with the first goal: demonstrating that the software meets its requirements.

- In validation testing, the expectation is that the system will perform correctly when given a specific set of test cases that mirror the system's anticipated use.
- A successful test in validation testing indicates that the system operates as intended.
- **Defect Testing:**
 - This type of testing aligns with the second goal: discovering faults or defects where the software's behavior is incorrect or does not conform to its specification.
 - Test cases for defect testing are intentionally designed to expose defects.
 - These test cases can be deliberately obscure and do not necessarily need to reflect how the system is typically used.
 - Defect testing focuses on identifying undesirable system behaviors such as system crashes, unwanted interactions with other systems, or incorrect computations.
 - A successful test in defect testing is one that causes the system to perform incorrectly, thereby exposing a defect.

Verification vs. Validation (V&V)

The terms verification and validation represent different aspects of ensuring software quality:

- **Verification:** This answers the question, "Are we building the product right?". It focuses on whether the software conforms to its specification.
- **Validation:** This answers the question, "Are we building the right product?". It focuses on whether the software truly addresses what the user requires.

V&V Confidence

The ultimate aim of Verification and Validation (V&V) is to establish confidence that the system is 'fit for purpose'. The level of confidence required and the thoroughness of V&V activities depend on several factors:

- **Software Purpose:** The criticality of the software to an organization directly influences the required level of confidence. Highly critical systems demand higher confidence.
- **User Expectations:** Users may have varying expectations for different types of software. For instance, expectations might be lower for experimental or non-critical applications.
- **Marketing Environment:** In some cases, getting a product to market quickly might be prioritized over finding every possible defect, especially in competitive environments.

Inspections and Testing

Software inspections and software testing are two complementary, not opposing, verification techniques that should both be utilized during the V&V process.

- **Software Inspections:**
 - These are concerned with the analysis of the static system representation to discover problems. This is also known as **static verification**.

- Inspections involve people meticulously examining the source representation (e.g., requirements, design, source code, configuration data, test data) with the goal of identifying anomalies and defects.
- A significant advantage is that inspections do not require the execution of a system, meaning they can be performed before implementation.
- Inspections may be supplemented by tool-based document and code analysis.
- **Software Testing:**
 - This is concerned with exercising and observing the product's behavior. This is known as **dynamic verification**.
 - The system is executed with specific test data, and its operational behavior is then observed.

Advantages of Inspections

Inspections offer several distinct advantages:

- **Error Masking:** During testing, one error can often mask or hide other errors, making them difficult to detect. Because inspection is a static process, this interaction between errors is not a concern.
- **Incomplete Versions:** Incomplete versions of a system can be inspected without incurring additional costs. In contrast, testing an incomplete program would necessitate developing specialized test harnesses to isolate and test the available parts.
- **Broader Quality Attributes:** Beyond just searching for program defects, an inspection can also assess broader quality attributes of a program, such as its compliance with standards, portability, and maintainability.

Limitations of Inspections

While valuable, inspections have limitations:

- Inspections can effectively check conformance with a specification, but they cannot verify conformance with the customer's real, often implicit, requirements.
- Inspections are unable to check non-functional characteristics of the system, such as performance, usability, or reliability. These attributes can only be assessed through dynamic testing.

Stages of Testing

The software testing process typically involves several distinct stages:

1. **Development Testing:** This occurs during the software development phase, with the aim of discovering bugs and defects.
2. **Release Testing:** A dedicated testing team tests a complete version of the system before it is released to users.
3. **User Testing:** Users or potential users of the system test it within their own operational environment.

Development Testing

Development testing encompasses all testing activities performed by the software development team. It includes three main levels:

1. **Unit Testing:**

- This involves testing individual program units or object classes in isolation.
- Unit testing is primarily a **defect testing process**.
- The focus of unit testing should be on validating the functionality of individual objects or methods.
- Units can refer to:
 - Individual functions or methods within an object.
 - Object classes that include several attributes and methods.
 - Composite components possessing defined interfaces for accessing their functionality.
- **Object Class Testing:**
 - Achieving complete test coverage for an object class involves:
 - Testing all operations associated with the object.
 - Setting and interrogating all attributes of the object.
 - Exercising the object in all its possible states.
 - Inheritance introduces complexity in designing object class tests because the information to be tested is not localized within a single class.
- **Weather Station Testing Example:**
 - For a weather station object, test cases would need to be defined for operations like `reportWeather`, `calibrate`, `test`, `startup`, and `shutdown`.
 - A state model can be used to identify sequences of state transitions to be tested and the event sequences required to trigger these transitions.
 - **Examples of state transition sequences to test:**
 - `Shutdown -> Running -> Shutdown`
 - `Configuring -> Running -> Testing -> Transmitting -> Running`
 - `Running -> Collecting -> Running -> Summarizing -> Transmitting -> Running`
- **Automated Testing:**
 - Whenever feasible, unit testing should be automated to allow tests to run and be checked without manual intervention.
 - Automated unit testing typically utilizes a test automation framework (e.g., JUnit) to write and execute program tests.
 - These frameworks provide generic test classes that developers extend to create specific test cases. They then run all implemented tests and report their success or failure, often through a graphical user interface (GUI).
- **Automated Test Components:**
 - Automated tests are composed of three main parts:
 - **Setup part:** This is where the system is initialized with the test case, including inputs and expected outputs.
 - **Call part:** This involves calling the specific object or method that is being tested.
 - **Assertion part:** Here, the actual result of the call is compared with the expected result. If the assertion evaluates to `true`, the test is successful; if `false`, it has failed.
- **Choosing Unit Test Cases:**
 - Test cases should demonstrate that the component functions as expected when used normally.
 - They should also reveal any defects present in the component.

- This leads to two primary types of unit test cases:
 - **Normal operation test cases:** These reflect the expected use of the program and confirm that the component works as anticipated.
 - **Abnormal input test cases:** These are based on common problems identified from testing experience. They use unusual or erroneous inputs to ensure proper processing and to prevent component crashes.
- **Testing Strategies:**
 - **Partition testing:** This involves identifying groups of inputs that share common characteristics and are expected to be processed identically. Tests should be selected from within each of these groups. These groups are often called **equivalence partitions** or **domains**, where the program behaves equivalently for all members of a class.
 - **Guideline-based testing:** This strategy uses predefined testing guidelines to select test cases. These guidelines are typically derived from past experience regarding common errors made by programmers during component development.
- **Testing Guidelines (Sequences):**
 - Test software with sequences containing only a single value.
 - Use sequences of varying sizes in different tests.
 - Derive tests to access the first, middle, and last elements of a sequence.
 - Test with sequences of zero length.
- **General Testing Guidelines:**
 - Choose inputs that force the system to generate all possible error messages.
 - Design inputs that cause input buffers to overflow.
 - Repeat the same input or series of inputs numerous times.
 - Force the generation of invalid outputs.
 - Force computation results to be either too large or too small.
- 2. **Component Testing:**
 - Software components are frequently composite, meaning they are built from several interacting objects. For example, in a weather station system, a reconfiguration component might include objects handling various aspects of reconfiguration.
 - The functionality of these objects is accessed through a defined component interface.
 - Therefore, testing composite components should primarily focus on ensuring that the component interface behaves according to its specification. It is generally assumed that unit tests on individual objects have already been conducted.
 - **Interface Testing:**
 - The main objective of interface testing is to detect faults that arise from interface errors or incorrect assumptions made about interfaces.
 - **Interface Types:**
 - **Parameter interfaces:** Data is passed between methods or procedures.
 - **Shared memory interfaces:** A block of memory is shared between procedures or functions.
 - **Procedural interfaces:** A sub-system encapsulates a set of procedures that can be called by other sub-systems.
 - **Message passing interfaces:** Sub-systems request services from other sub-systems by sending messages.
 - **Interface Errors:**
 - **Interface misuse:** Occurs when a calling component incorrectly uses the interface of another component (e.g., passing parameters in the wrong order).
 - **Interface misunderstanding:** Arises when a calling component makes incorrect assumptions about the behavior of the component it calls.

- **Timing errors:** Happen when the called and calling components operate at different speeds, leading to the access of out-of-date information.
- **Interface Testing Guidelines:**
 - Design tests where parameters passed to a called procedure are at the extreme ends of their permissible ranges.
 - Always test pointer parameters with null pointers.
 - Design tests that are intended to cause the component to fail.
 - In message passing systems, utilize stress testing to evaluate performance under heavy load.
 - In shared memory systems, vary the order in which components are activated to expose potential concurrency issues.
- 3. **System Testing:**
 - System testing during development involves integrating components to create a version of the complete system, which is then tested as a whole.
 - The primary focus of system testing is on examining the interactions between components.
 - It verifies that components are compatible, interact correctly, and transfer the right data at the appropriate time across their interfaces.
 - System testing is also crucial for testing the **emergent behavior** of a system, which are properties that arise from the interaction of components and are not evident in individual components.
 - During system testing, reusable components, separately developed components, and off-the-shelf systems may be integrated with newly developed components. The entire integrated system is then tested.
 - This stage is a collective process, not an individual one, as components from different team members or sub-teams are integrated. In some organizations, a separate testing team, independent of designers and programmers, may conduct system testing.
 - **Use-Case Testing:**
 - Use cases, developed to identify system interactions, can serve as a valuable basis for system testing.
 - Each use case typically involves multiple system components, and testing the use case forces these interactions to occur.
 - Sequence diagrams associated with the use case documents detail the components and interactions being tested.
 - **Test Cases Derived from Sequence Diagram (Mentcare System Example):**
 - Consider a scenario where a request for a report is sent to `WeatherStation` (via `SatComms`).
 - **Example Test Case 1:** An input request for a report should result in an associated acknowledgment, and ultimately, a report should be returned. You should create summarized data beforehand to verify that the report is correctly organized when returned.
 - **Example Test Case 2:** An input request for a report to the `WeatherStation` object should lead to a summarized report being generated. This can be tested by creating raw data that corresponds to the summary prepared for the `SatComms` test. Then, verify that the `WeatherStation` object correctly produces this summary. This raw data is also used to test the `WeatherData` object.
 - **Testing Policies:**
 - Exhaustive system testing is practically impossible. Therefore, testing policies are developed to define the required system test coverage.
 - **Examples of testing policies:**

- All system functions accessible via menus should be tested.
- Combinations of functions (e.g., text formatting) accessed through the same menu must be tested.
- Wherever user input is provided, all functions must be tested with both correct and incorrect input.

Test-Driven Development (TDD)

Test-driven development (TDD) is an approach to program development where testing and code development are interleaved.

- In TDD, tests are written *before* the code. The critical driver of development is to make the code 'pass' these pre-written tests.
- Code is developed incrementally, with a test created for each small increment of functionality. Development progresses to the next increment only after the current code successfully passes its test.
- TDD originated as a part of agile methodologies, such as Extreme Programming, but it can also be effectively used in plan-driven development processes.

TDD Process Activities

The TDD process follows a specific cycle:

1. **Identify Functionality Increment:** Begin by identifying a small increment of functionality that needs to be implemented. This should ideally be implementable in just a few lines of code.
2. **Write Test:** Write a test specifically for this new functionality and implement it as an automated test.
3. **Run Tests (Fail First):** Run the newly written test along with all other previously implemented tests. Since the functionality has not yet been implemented, the new test is expected to fail initially.
4. **Implement Functionality:** Implement the required functionality.
5. **Rerun Tests (Pass):** Rerun all tests.
6. **Refactor & Next Increment:** Once all tests run successfully, you can refactor the code if necessary and then move on to implementing the next chunk of functionality.

Benefits of Test-Driven Development

TDD offers several significant advantages:

- **Code Coverage:** Every segment of code written has at least one associated test, ensuring that all code is covered by tests.
- **Regression Testing:** A comprehensive regression test suite is incrementally built up as the program is developed.
 - **Regression testing** is the process of testing the system to ensure that changes made have not inadvertently 'broken' previously working code.
 - While expensive in manual testing, regression testing is simple and straightforward with automated tests. All tests can be rerun every time a change is made to the program.
 - Changes should only be committed once all tests run successfully.

- **Simplified Debugging:** When a test fails, the problem's location should be relatively obvious, typically within the newly written or recently modified code that needs checking.
- **System Documentation:** The tests themselves act as a form of executable documentation, illustrating how the system is intended to be used and how specific functionalities should behave.

Release Testing

Release testing is the process of testing a specific version of a system that is intended for use outside of the development team.

- The primary goal of release testing is to convince the system supplier that the software is robust and ready for use.
- Therefore, release testing must demonstrate that the system delivers its specified functionality, performance, and dependability, and that it does not fail during normal operation.
- Release testing is typically a **black-box testing** process, meaning tests are derived solely from the system specification without knowledge of the internal code structure.

Release Testing and System Testing

While release testing is a form of system testing, there are crucial differences:

- **Team Responsibility:** A separate team, distinct from the system development team, should be responsible for conducting release testing.
- **Objective:** System testing performed by the development team primarily focuses on discovering bugs and defects (defect testing). In contrast, the objective of release testing is to verify that the system meets its requirements and is of sufficient quality for external use (validation testing).

Requirements-Based Testing

Requirements-based testing involves examining each documented requirement and developing one or more test cases specifically for it.

- **Mentcare System Requirements Example:**
 - **Requirement 1:** "If a patient is known to be allergic to any particular medication, then prescription of that medication shall result in a warning message being issued to the system user."
 - **Requirement 2:** "If a prescriber chooses to ignore an allergy warning, they shall provide a reason why this has been ignored."
- **Requirements Tests (Derived from Mentcare Example):**
 - **Test Case 1 (No Allergy Warning):** Set up a patient record with no known allergies. Prescribe medication for allergies that are known to exist (but not for this patient). Check that no warning message is issued by the system.
 - **Test Case 2 (Single Allergy Warning):** Set up a patient record with a known allergy. Prescribe the medication to which the patient is allergic. Check that the warning message is issued by the system.

- **Test Case 3 (Multiple Separate Allergy Warnings):** Set up a patient record where allergies to two or more drugs are recorded. Prescribe each of these drugs separately. Check that the correct warning for each drug is issued.
- **Test Case 4 (Multiple Concurrent Allergy Warnings):** Prescribe two drugs to which the patient is allergic simultaneously. Check that two warnings are correctly issued.
- **Test Case 5 (Overruling Warning):** Prescribe a drug that triggers an allergy warning and then choose to override that warning. Check that the system prompts the user to provide information explaining why the warning was overruled.

A Usage Scenario for the Mentcare System

Usage scenarios can also be used to derive test cases, particularly for system testing and understanding complex interactions.

Scenario: George, a mental healthcare nurse, visits patients at home. He logs into the Mentcare system and prints his daily schedule with patient summaries. He downloads these records to his laptop, encrypting them with a key phrase. During a visit to Jim, a patient on depression medication, Jim reports sleeplessness. George decrypts Jim's record, checks the drug's side effects, finds sleeplessness listed, and notes the problem in Jim's record. He suggests a clinic visit to change medication and sets a prompt for himself to call Jim to make an appointment. After the consultation, Jim's record is re-encrypted. Back at the clinic, George uploads the visited patient records. The system generates a call list for follow-up and appointment scheduling.

- **Features Tested by Scenario:** This scenario implicitly tests several system features:
 - Authentication by logging on to the system.
 - Downloading and uploading of specified patient records to a laptop.
 - Home visit scheduling functionality.
 - Encryption and decryption of patient records on a mobile device.
 - Record retrieval and modification.
 - Links with the drugs database that maintains side-effect information.
 - The system for call prompting.

Performance Testing

Performance testing is a component of release testing that evaluates the emergent properties of a system, such as its performance and reliability.

- Tests should accurately reflect the expected usage profile of the system.
- Typically, performance tests involve planning a series of tests where the system's load is progressively increased until its performance becomes unacceptable.
- **Stress testing** is a specific form of performance testing where the system is intentionally overloaded beyond its normal capacity to observe its failure behavior.

User Testing

User or customer testing is a critical stage in the testing process where actual users or potential customers provide input and advice on the system's testing.

- User testing is essential even after extensive system and release testing.
- The primary reason for its necessity is that influences from the user's actual working environment significantly affect a system's reliability, performance, usability, and robustness. These environmental factors cannot be accurately replicated in a controlled testing environment.

Types of User Testing

There are three main types of user testing:

1. **Alpha Testing:**
 - Users of the software collaborate with the development team to test the software at the developer's site. This provides immediate feedback and a controlled environment.
2. **Beta Testing:**
 - A release of the software is made available to a wider group of users who are allowed to experiment with it in their own environments. These users then report any problems they discover directly to the system developers.
3. **Acceptance Testing:**
 - Customers test a system to determine whether it is ready to be formally accepted from the system developers and deployed into their operational environment.
 - This type of testing is primarily used for custom-developed systems.

The Acceptance Testing Process

The acceptance testing process typically involves the following stages:

1. Define acceptance criteria.
2. Plan acceptance testing.
3. Derive acceptance tests.
4. Run acceptance tests.
5. Negotiate test results with the development team.
6. Make a decision to reject or accept the system.

Agile Methods and Acceptance Testing

In agile development methodologies:

- The user or customer is considered an integral part of the development team.
- This user/customer is responsible for making decisions regarding the system's acceptability.
- Tests, defined by the user/customer, are integrated with other tests and are run automatically whenever changes are made to the system.
- Consequently, there is no separate acceptance testing process in many agile contexts.
- A key challenge with this approach is ensuring that the embedded user is truly 'typical' and can accurately represent the interests of all system stakeholders.

Key Points

- Testing can only demonstrate the presence of errors in a program; it cannot prove the absence of remaining faults.
- Development testing is the responsibility of the software development team, while a separate team should handle testing a system before its release to customers.
- Development testing comprises unit testing (individual objects/methods), component testing (groups of objects), and system testing (partial or complete systems).
- When testing software, it is advisable to actively try to 'break' the software by applying experience and guidelines to select test cases that have proven effective in discovering defects in other systems.
- Where possible, write automated tests that are embedded in a program and can be run every time a system change is made.
- Test-first development is an approach where tests are written *before* the code they are intended to test.
- Scenario testing involves creating a typical usage scenario and deriving test cases from it.
- Acceptance testing is a user testing process whose objective is to decide if the software is fit for deployment and use in its operational environment.

UNIT-III: Software Testing, Software Evolution, Component Based Software Engineering

Chapter 9 – Software Evolution

Topics covered:

- Evolution processes
- Legacy systems
- Software maintenance

Software Change

- Software change is an inevitable aspect of software systems.
- **Reasons for Software Change:**
 - New requirements emerge as the software is used.
 - The business environment undergoes changes.
 - Errors within the software must be repaired.
 - New computers and equipment are added to the system, necessitating adjustments.
 - The performance or reliability of the system may require improvement.
- A significant challenge for all organizations is the implementation and management of changes to their existing software systems.

Importance of Evolution

- Organizations have substantial financial investments in their software systems, as these systems are critical business assets.
- To preserve and maintain the value of these assets to the business, the software systems must be continuously changed and updated.

- The majority of the software budget in large companies is allocated to changing and evolving existing software, rather than developing new software.

A Spiral Model of Development and Evolution

Evolution and Servicing

- **Evolution:** This is a stage in a software system's life cycle where the system is actively in operational use and undergoes continuous changes as new requirements are proposed and subsequently implemented.
- **Servicing:** During this stage, the software remains functional and useful, but the only changes made are those essential to keep it operational. This primarily includes bug fixes and modifications necessary to adapt to changes in the software's operating environment. No new functionality is added during this stage.
- **Phase-out:** In this final stage, the software may still be in use, but no further changes or modifications are made to it.

Evolution Processes

- Software evolution processes are highly dependent on several factors:
 - The specific type of software being maintained.
 - The particular development processes that were used.
 - The skills and experience of the individuals involved in the evolution process.
- **Driver for System Evolution:** Proposals for change act as the primary driver for system evolution.
 - These change proposals should be explicitly linked with the components of the system that will be affected by the proposed change.
 - This linkage allows for a more accurate estimation of the cost and impact that the change will have on the system.
- The continuous process of identifying changes and evolving the software persists throughout the entire lifetime of the system.

Change Identification and Evolution Processes

The Software Evolution Process

Change Implementation

- Change implementation represents an iteration of the development process, during which revisions to the system are designed, implemented, and thoroughly tested.
- **Critical Difference:** A key distinction in change implementation is that the initial stage may involve "program understanding". This is particularly crucial if the original developers of the system are not the ones responsible for implementing the change.
- **Program Understanding Phase:** During this phase, the implementers must acquire a clear understanding of:
 - How the program is fundamentally structured.
 - How the program delivers its intended functionality.
 - How the proposed change might potentially affect the existing program structure and functionality.

Urgent Change Requests

- In certain situations, urgent changes may need to be implemented without following all the conventional stages of the software engineering process.
- **Scenarios requiring urgent changes:**
 - When a serious system fault must be repaired immediately to restore normal operational continuity.
 - If changes made to the system's environment, such as an operating system upgrade, result in unexpected and critical effects.
 - When business changes demand a very rapid response, for instance, the release of a competing product.
- An emergency repair process exists for such situations.

Agile Methods and Evolution

- Agile methods are inherently based on an incremental development approach, which facilitates a seamless transition from the development phase to the evolution phase.
- For agile processes, evolution is simply a continuous extension of the development process, characterized by frequent system releases.
- **Automated Regression Testing:** This practice is especially valuable when modifications are made to a system, as it helps ensure that new changes do not break existing functionality.
- Changes within an agile context may be articulated as additional user stories.

Handover Problems

- **Scenario 1: Agile Development, Plan-Based Evolution:**
 - This problem arises when the software development team has used an agile approach, but the evolution (maintenance) team is unfamiliar with agile methods and prefers a more traditional, plan-based approach.
 - The evolution team may anticipate and require detailed documentation to support their work, which is typically not produced in the same comprehensive manner in agile processes.
- **Scenario 2: Plan-Based Development, Agile Evolution:**
 - Conversely, handover problems can occur when a plan-based approach was used for development, but the evolution team prefers to use agile methods.
 - In this situation, the agile evolution team may need to develop automated tests from scratch.
 - Additionally, the code in the system may not have undergone refactoring and simplification, which is an expected practice in agile development for maintaining code quality.

Legacy Systems

- **Definition:** Legacy systems are older software systems that depend on programming languages and technologies that are no longer actively used for new systems development.
- These systems may also be dependent on older hardware, such as mainframe computers.

- Furthermore, they can have associated legacy processes and procedures that were designed around the older technology.
- Legacy systems are not exclusively software systems; they are considered broader socio-technical systems.
- **Components of a Socio-Technical Legacy System:**
 - **Hardware:** This includes the physical computing infrastructure. Legacy systems may have been originally developed for hardware that is no longer readily available or supported.
 - **Software:** This encompasses the operating systems, libraries, and other supporting software components. The legacy system may rely on a range of support software that could be obsolete or no longer actively supported by vendors.
 - **Application Software:** This refers to the core application system that provides the essential business services. It typically comprises multiple application programs.
 - **Application Data:** These are the data processed by the application system. Legacy data can often be inconsistent, duplicated across different storage locations, or held in disparate databases.
 - **Business Processes:** These are the established procedures used within the business to achieve specific objectives. Business processes may have been designed around, and are often constrained by, the specific functionality provided by the legacy system.
 - **Business Policies and Rules:** These define how business operations should be conducted and impose constraints on the business. The usage of the legacy application system can be deeply embedded within these policies and rules.

Legacy System Layers

Legacy System Replacement

- Replacing a legacy system is inherently risky and expensive, which is why businesses frequently choose to continue using these older systems.
- **Reasons for High Risk in System Replacement:**
 - **Lack of Complete System Specification:** Comprehensive and accurate documentation of the original system's requirements may be missing.
 - **Tight Integration of System and Business Processes:** The legacy system is often deeply intertwined with critical business processes, making separation difficult.
 - **Undocumented Business Rules:** Many business rules might be implicitly embedded within the legacy system's code rather than formally documented.
 - **New Software Development Challenges:** New software development projects carry inherent risks of being delivered late and/or exceeding their allocated budget.

Legacy System Change

- Legacy systems are generally expensive and challenging to modify for several reasons:

- **No Consistent Programming Style:** The code may have been developed over many years by different people, leading to varied and inconsistent programming styles.
- **Use of Obsolete Programming Languages:** The system may be written in programming languages that are no longer widely used, resulting in a scarcity of skilled personnel proficient in those languages.
- **Inadequate System Documentation:** Comprehensive and up-to-date documentation for the system may be lacking.
- **System Structure Degradation:** Over time, repeated modifications can degrade the system's architectural structure, making further changes more difficult.
- **Program Optimizations:** Older programs might contain low-level optimizations that make them extremely difficult to understand for modern developers.
- **Data Errors, Duplication, and Inconsistency:** The data processed by legacy systems often suffers from errors, duplication, and inconsistencies, complicating data migration or integration with new systems.

Legacy System Management

- Organizations that rely on legacy systems must strategically choose a plan for evolving these systems.
- **Strategies for Legacy System Evolution:**
 - **Scrap the System Completely:** This involves discontinuing the use of the legacy system entirely and modifying existing business processes so that the system is no longer required.
 - **Continue Maintaining the System:** The organization opts to keep the system in operation and perform necessary maintenance activities to ensure its continued functionality.
 - **Transform the System by Re-engineering:** This strategy involves re-engineering the system to improve its maintainability, often by restructuring or rewriting parts of it without changing its core functionality.
 - **Replace the System with a New System:** The organization develops or acquires a completely new system to take over the functions performed by the legacy system.
- **Strategy Selection:** The choice of strategy should be based on an assessment of both the system's quality and its business value.

Figure 9.13 An example of a legacy system assessment

Legacy System Categories

- Based on their quality and business value, legacy systems can be categorized to guide management strategy:
 - **Low Quality, Low Business Value:** These systems should be scrapped as they provide little business benefit and are problematic to maintain.
 - **Low-Quality, High-Business Value:** These systems are critical to business operations but are expensive and problematic to maintain due to their low quality. They should ideally be re-engineered to improve maintainability or replaced if a suitable alternative system is available.

- **High-Quality, Low-Business Value:** These systems are well-structured and easy to maintain but offer limited business contribution. Options include replacing them with commercial off-the-shelf (COTS) solutions, scrapping them entirely, or continuing to maintain them if the cost is minimal.
- **High-Quality, High Business Value:** These systems are valuable business assets and are relatively easy to maintain. They should continue in operation, undergoing normal system maintenance.

Business Value Assessment

- Assessment of business value should incorporate different viewpoints to provide a comprehensive understanding.
- **Viewpoints to consider:**
 - System end-users.
 - Business customers.
 - Line managers.
 - IT managers.
 - Senior managers.
- The assessment process typically involves interviewing various stakeholders and then consolidating the collected results.

Issues in Business Value Assessment

- **The Use of the System:** If systems are only used occasionally or by a small number of people, their overall business value may be considered low.
- **The Business Processes that are Supported:** A system may have a low business value if its design or limitations force the business to use inefficient processes.
- **System Dependability:** If a system is unreliable and its problems directly impact business customers, then the system possesses a low business value.
- **The System Outputs:** If the business critically depends on the outputs generated by the system, then the system itself holds high business value.

System Quality Assessment

- To evaluate the quality of a legacy system, assessment should be conducted across three main areas:
 - **Business Process Assessment:** How effectively does the business process supported by the system align with the current goals of the business?
 - **Environment Assessment:** How effective is the system's operating environment, and how expensive is it to maintain?
 - **Application Assessment:** What is the quality of the application software system itself?

Business Process Assessment

- This assessment should use a viewpoint-oriented approach, seeking answers directly from system stakeholders.
- **Key Questions for Business Process Assessment:**

- Is there a clearly defined process model, and is it consistently followed within the organization?
- Do different parts of the organization utilize different processes for performing the same function?
- How has the business process been adapted over time?
- What are the relationships between this process and other business processes, and are these relationships necessary?
- Is the process adequately and effectively supported by the existing legacy application software?
- **Example:** A travel ordering system might have a low business value because of the widespread and more efficient use of modern web-based ordering platforms.

Factors Used in Environment Assessment

Factor	Questions	Source
Supplier stability	Is the supplier of the hardware or software still in existence? Is the supplier financially stable and likely to continue in existence to provide support? If the original supplier is no longer in business, is there another entity or community that maintains the systems or provides support?	
Failure rate	Does the hardware exhibit a high rate of reported failures, leading to downtime or operational issues? Does the support software frequently crash, necessitating system restarts and disrupting operations?	
Age	How old are the underlying hardware and support software components of the system? Older hardware and support software are more likely to be obsolete. While they may still function correctly, there could be significant economic and business benefits (e.g., lower maintenance costs, improved performance) from migrating to a more modern system.	
Performance	Is the current performance of the system adequate to meet business needs and user expectations? Do performance problems (e.g., slow response times, batch processing delays) have a significant negative effect on system users or business operations?	
Support requirements	What level of local support (e.g., dedicated IT staff, specialized technicians) is required for the hardware and software components of the legacy system? If there are high costs associated with providing this local support, it may be a strong indicator to consider system replacement as a more economical long-term solution.	
Maintenance costs	What are the recurring costs associated with hardware maintenance and the licensing fees for support software? Older hardware often incurs higher maintenance costs compared to modern systems. Similarly, support software may have substantial annual licensing costs that make continued use expensive.	
Interoperability	Are there significant problems or challenges in interfacing the legacy system with other modern systems or applications? Can compilers, for example, be effectively	

	used with current versions of the operating system? Is hardware emulation required to run the legacy system, adding complexity and potential performance overhead?	
--	--	--

Factors Used in Application Assessment

Factor	Questions	Source
Understandability	How difficult is it for developers or maintainers to comprehend the source code of the current system? What is the level of complexity in the control structures (e.g., intricate loops, nested conditionals) used throughout the code? Do variables have meaningful, descriptive names that accurately reflect their function, or are they cryptic?	
Documentation	What system documentation (e.g., design documents, user manuals, code comments) is currently available for the application? Is the existing documentation complete, consistent across different sections, and up-to-date with the current state of the system?	
Data	Is there an explicit and well-defined data model (e.g., database schema) for the system that clearly describes its data structures? To what extent is data duplicated across various files, databases, or modules within the system? Is the data actively used by the system current, accurate, and consistent across all its instances?	
Performance	Is the overall performance of the application adequate to meet user demands and business requirements (e.g., response times, throughput)? Do performance problems (e.g., slow data processing, long report generation) have a significant negative impact on system users or business operations?	
Programming language	Are modern compilers or development environments readily available for the programming language in which the system was developed? Is the programming language still actively used and supported for new system development projects within the organization or industry?	
Configuration management	Are all versions of all parts of the system (e.g., source code, documentation, configuration files) effectively managed and controlled by a robust configuration management system? Is there a clear and explicit description of the specific versions of components that are integrated and used in the current operational system?	
Test data	Does comprehensive test data for the system exist, which can be used to validate changes and ensure correct functionality? Is there a documented record of regression tests that have been carried out whenever new features or modifications have been added to the system, to prevent unintended side effects?	
Personnel skills	Are there adequately skilled personnel available within the organization or external market who possess the necessary	

	expertise to maintain the application, including knowledge of its programming language, architecture, and domain? Are there individuals available who have prior experience working with this specific system, understand its nuances, and can provide institutional knowledge?	
--	---	--

System Measurement

- Quantitative data can be collected to objectively assess the quality of an application system.
- **Metrics for Quality Assessment:**
 - **The number of system change requests:** A higher accumulated value for change requests generally indicates a lower quality of the system, suggesting more issues or required modifications.
 - **The number of different user interfaces used by the system:** The presence of a greater number of disparate interfaces increases the likelihood of inconsistencies and redundancies across these interfaces, impacting usability and maintainability.
 - **The volume of data used by the system:** As the amount of data processed by the system (e.g., number of files, size of database) increases, so does the potential for inconsistencies and errors within that data.
 - Cleaning up old and problematic data is typically a very expensive and time-consuming process.

Software Maintenance

- **Definition:** Software maintenance refers to the process of modifying a program after it has been deployed and put into operational use.
- The term "maintenance" is predominantly used for changes made to custom software systems.
- In contrast, generic software products are typically described as "evolving" to create new versions.
- Maintenance activities generally do not involve significant changes to the system's fundamental architecture.
- Changes are implemented by modifying existing software components and, where necessary, adding new components to the system.

Types of Maintenance

- **Fault Repairs (Corrective Maintenance):** This type of maintenance involves changing a system to fix identified bugs, address security vulnerabilities, and correct any deficiencies in how the system meets its original requirements.
- **Environmental Adaptation (Adaptive Maintenance):** This involves modifying the software to enable it to operate effectively in a different or updated operating environment. This includes changes required due to new computer hardware, operating system upgrades, or other environmental shifts from its initial implementation.
- **Functionality Addition and Modification (Perfective Maintenance):** This type of maintenance focuses on modifying the existing system to satisfy new or changed requirements, thereby enhancing or expanding its capabilities.

Maintenance Effort Distribution

Maintenance Costs

- **Typical Costs:** Maintenance costs are usually significantly higher than the initial development costs, often ranging from 2 to 100 times the development cost, depending on the specific application.
- **Influencing Factors:** Maintenance costs are affected by both technical aspects (e.g., code quality) and non-technical factors (e.g., team experience).
- **Cost Escalation:** Maintenance costs tend to increase as software continues to be maintained over time.
 - This is because the maintenance process itself can gradually corrupt the software's original structure, making subsequent maintenance activities more difficult and costly.
- **Ageing Software:** Older software systems can incur particularly high support costs due to factors like reliance on old programming languages, outdated compilers, or lack of vendor support.
- **Adding Features:** It is typically more expensive to add new features to an existing system during the maintenance phase than it would have been to incorporate those same features during the initial development phase.
- **Reasons for Higher Maintenance Costs (detailed):**
 - A new team, often different from the original developers, must invest time to understand the existing programs being maintained.
 - When development and maintenance are separated, there may be little incentive for the initial development team to write highly maintainable software.
 - Program maintenance work is generally unpopular among software engineers, which can lead to lower morale or less experienced staff being assigned.
 - Maintenance staff are often less experienced and may possess limited domain knowledge compared to the original development team.
 - As programs age, their internal structure tends to degrade due to repeated changes, making them progressively harder to modify.

Maintenance Prediction

- Maintenance prediction aims to identify which parts of a software system are likely to cause problems and incur high maintenance costs in the future.
- **Key Relationships in Prediction:**
 - The acceptance of a change request is dependent on the maintainability of the specific components that are affected by that change.
 - The act of implementing changes can degrade the system's structure, thereby reducing its overall maintainability.
 - Overall maintenance costs are directly related to the total number of changes required, and the cost of implementing each change is influenced by the system's maintainability.

Change Prediction

- Predicting the number of changes a system will require necessitates a thorough understanding of the intricate relationships between the system and its operational environment.
- **Tightly Coupled Systems:** Systems that are tightly coupled with their environment will typically require changes whenever the environment itself undergoes modification.
- **Factors Influencing this Relationship:**
 - The number and inherent complexity of the system interfaces.
 - The number of system requirements that are intrinsically volatile (prone to frequent changes).
 - The specific business processes in which the system is actively used.

Complexity Metrics

- Predictions regarding a system's maintainability can be made by quantitatively assessing the complexity of its individual components.
- Research studies have consistently shown that the majority of software maintenance effort is concentrated on a relatively small number of system components, which are typically the most complex ones.
- **Factors Contributing to Complexity:**
 - The complexity of control structures (e.g., nested loops, conditional statements) within the code.
 - The complexity of data structures used (e.g., intricate object hierarchies, complex database schemas).
 - The size of objects, methods (procedures), and modules within the system.

Process Metrics

- Process metrics, which describe characteristics of the software development and maintenance process, can also be utilized to assess a system's maintainability.
- **Examples of Process Metrics for Maintainability:**
 - The total number of requests for corrective maintenance.
 - The average time required to perform an impact analysis for a change request.
 - The average time taken to fully implement a change request.
 - The number of outstanding (unaddressed) change requests at any given time.
- If any or all of these process metrics show an increasing trend, it may serve as an indicator of a decline in the system's overall maintainability.

Software Reengineering

- **Definition:** Software reengineering is the process of restructuring or rewriting part or all of a legacy system without altering its fundamental functionality.
- **Applicability:** This approach is particularly applicable in situations where some, but not all, sub-systems of a larger system require frequent and costly maintenance.
- Reengineering involves investing additional effort to make the system components easier to maintain in the future.
- During reengineering, the system may be restructured (e.g., improved modularity) and re-documented to enhance its comprehensibility.

Advantages of Reengineering

- **Reduced Risk:** New software development projects inherently carry high risks, including potential development problems, staffing challenges, and issues with incomplete or changing specifications. Reengineering mitigates many of these risks because the core functionality is already established.
- **Reduced Cost:** The cost associated with re-engineering an existing system is often significantly less than the costs involved in developing a completely new software system from scratch.

The Reengineering Process

Reengineering Process Activities

- **Source Code Translation:** This activity involves converting the existing source code from one programming language to a newer or more supported language.
- **Reverse Engineering:** This process involves analyzing the existing program to understand its design, structure, and functionality, especially when documentation is poor or nonexistent.
- **Program Structure Improvement:** This activity focuses on automatically restructuring the program to enhance its understandability, often by applying refactoring techniques.
- **Program Modularization:** This involves reorganizing the program's structure into more coherent, independent modules to improve its maintainability and reusability.
- **Data Reengineering:** This critical activity involves cleaning up inconsistent or duplicated data and restructuring the system's data models to improve consistency and efficiency.

Reengineering Approaches

Reengineering Cost Factors

- The overall cost of reengineering is influenced by several factors:
 - **Quality of the Software:** The initial quality of the software to be reengineered plays a significant role; lower quality (e.g., spaghetti code) will likely incur higher reengineering costs.
 - **Tool Support:** The availability and effectiveness of automated tools for reengineering (e.g., code analysis tools, translation tools) can significantly impact cost.
 - **Extent of Data Conversion:** If extensive data conversion or restructuring is required, it will add substantially to the reengineering cost.
 - **Availability of Expert Staff:** The presence of expert staff with knowledge of both the old technology and the new target technologies is crucial. This can be a significant problem, especially with very old systems based on technology that is no longer widely used or understood.

Refactoring

- **Definition:** Refactoring is a process of making internal improvements to a program's structure or design without changing its external behavior or functionality. Its primary goal is to slow down the degradation of the software's structure that occurs through continuous changes.
- Refactoring can be considered a form of "preventative maintenance" because it proactively reduces the problems and difficulties associated with future changes to the system.
- It involves modifying a program to enhance its structure, reduce its inherent complexity, or make it generally easier to understand.
- **Key Principle:** When performing refactoring, the focus should strictly be on improving the program's internal quality, not on adding new functionality.

Refactoring and Reengineering

- **Re-engineering:** This typically occurs after a system has been in use and maintained for a considerable period, and its maintenance costs begin to escalate significantly. It often involves using automated tools to process and re-engineer a legacy system to produce a new system that is more maintainable.
- **Refactoring:** This is presented as a continuous process of improvement that takes place throughout both the development and evolution phases of a software system. Its purpose is to actively prevent or mitigate the gradual degradation of the software's structure and code quality, which otherwise leads to increased costs and difficulties in maintaining the system over time.

‘Bad Smells’ in Program Code

- "Bad smells" refer to certain code structures that suggest a deeper problem in the design and indicate areas ripe for refactoring.
- **Common 'Bad Smells':**
 - **Duplicate Code:** This occurs when the same or very similar blocks of code appear in multiple locations within a program.
 - **Refactoring Solution:** This duplicated code can be extracted and implemented as a single, reusable method or function that is then called from the various places where it was duplicated.
 - **Long Methods:** If a method (or function) is excessively long, it often indicates it is performing too many responsibilities.
 - **Refactoring Solution:** Such a method should be redesigned and broken down into a number of shorter, more focused methods, each handling a single responsibility.
 - **Switch (case) Statements:** These often lead to duplication of logic, particularly when the `switch` statement's behavior depends on the type of a value. The different `case` branches can be scattered throughout a program.
 - **Refactoring Solution (in Object-Oriented Languages):** In object-oriented programming, this situation can often be elegantly resolved by leveraging polymorphism, where different object types handle the specific variations of the behavior.
 - **Data Clumping:** Data clumps occur when the same group of data items (e.g., fields within classes, parameters passed to methods) repeatedly appear together in multiple places across a program.

- **Refactoring Solution:** These recurring data clumps can often be encapsulated within a new object, which then represents and manages all of that related data.
- **Speculative Generality:** This "bad smell" arises when developers introduce excessive generality into a program, adding features or structures "just in case" they might be needed in the future, even if there's no immediate requirement.
 - **Refactoring Solution:** Such unnecessary generality can often simply be removed, simplifying the code and making it easier to maintain.

UNIT-III: Software Testing, Software Evolution, Component Based Software Engineering

This unit covers several key areas in software engineering, including Software Testing, Software Evolution, and Component Based Software Engineering.

Chapter 16 - Component-based software engineering

This chapter specifically delves into:

- Components and component models.
- CBSE processes.
- Component composition.

Component-based Development

Definition of Component-based Software Engineering (CBSE) CBSE is a software development approach that fundamentally relies on the reuse of entities known as 'software components'.

Emergence of CBSE CBSE emerged due to the limitations of object-oriented development in supporting effective software reuse. Object-oriented development often dealt with single object classes, which were found to be too detailed and specific for broad reuse. Components, in contrast, are more abstract than object classes and are designed to function as stand-alone service providers. They can exist independently as self-contained entities.

CBSE Essentials The successful implementation of CBSE hinges on several essential elements:

- **Independent Components:** Components must be independent and clearly specified by their interfaces.
- **Component Standards:** Standards are necessary to facilitate the smooth integration of different components.
- **Middleware Support:** Middleware is crucial as it provides the necessary infrastructure for component interoperability.
- **Development Process Geared to Reuse:** The entire development process must be structured to actively promote and support reuse.

CBSE and Design Principles Beyond the direct benefits of reuse, CBSE is grounded in robust software engineering design principles, which include:

- **Independence:** Components are designed to be independent, meaning they do not interfere with each other.
- **Information Hiding:** The internal implementation details of components are hidden from external view.
- **Well-Defined Interfaces:** Communication between components occurs exclusively through clearly defined interfaces.
- **Replaceability:** If a component's interface is maintained, one component can be replaced by another without affecting the rest of the system.
- **Standard Services:** Component infrastructures offer a variety of standard services that components can utilize.

Component Standards Establishing standards is vital for components to communicate and inter-operate effectively. However, historically, the adoption of CBSE has been hindered by the existence of several competing component standards. Examples include:

- Sun's Enterprise Java Beans (EJB).
- Microsoft's COM and .NET.
- CORBA's CCM (Corba Component Model). The practical consequence of these multiple, incompatible standards is that components developed using different approaches cannot work together, limiting widespread interoperability.

Service-oriented Software Engineering Service-oriented software engineering (covered in Chapter 18) shares similarities with CBSE. An executable service is considered a type of independent component. A key distinction is that services typically have a 'provides' interface but lack a 'requires' interface. From their inception, services have been built around common standards, which largely eliminates the communication problems encountered with different vendors' services in traditional CBSE. While system performance with services might be slower, this approach is increasingly replacing traditional CBSE in many systems.

Components and Component Models

Components Components are designed to provide a service regardless of where they are executing or the programming language they are written in.

- A component is an independent executable entity that can be composed of one or more executable objects.
- The component's interface is published, and all interactions with the component occur exclusively through this published interface.

Component Definitions

- **Councill and Heinmann:** "A software component is a software element that conforms to a component model and can be independently deployed and composed without modification according to a composition standard".
- **Szyperski:** "A software component is a unit of composition with contractually specified interfaces and explicit context dependencies only. A software component can be deployed independently and is subject to composition by third-parties".

Component Characteristics Components possess specific characteristics that enable their reusability and deployment:

Component Characteristic	Description
Composable	For a component to be composable, all external interactions must occur through publicly defined interfaces. Additionally, it must provide external access to information about itself, such as its methods and attributes, allowing it to be integrated into larger systems.
Deployable	To be deployable, a component must be self-contained and capable of operating as a stand-alone entity on a component platform that provides an implementation of the component model. This often implies that the component is in a binary form and does not require compilation before deployment. If a component is implemented as a service, it is deployed by the service provider, not by the user of the component.
Documented	Components require thorough documentation. This documentation allows potential users to determine if the component meets their specific needs. Ideally, both the syntax and semantics of all component interfaces should be clearly specified.
Independent	A component should be independent, meaning it can be composed and deployed without requiring the use of other specific components. If a component needs externally provided services, these requirements must be explicitly stated in a 'requires' interface specification.
Standardized	Component standardization dictates that a component used in a CBSE process must conform to a predefined standard component model. This model typically defines aspects like component interfaces and other component elements.

Component as a Service Provider A component functions as an independent, executable entity, meaning it does not need to be compiled before being used with other components. The services it offers are exposed through an interface, and all interactions with the component occur strictly through this interface. The component interface is defined in terms of parameterized operations, and crucially, its internal state is never directly exposed to external users.

Component Interfaces Components typically have two main types of interfaces:

- **Provides Interface:** This interface defines the services that the component offers to other components. It essentially acts as the component's Application Programming Interface (API), outlining the methods that can be invoked by a user of the component.
- **Requires Interface:** This interface specifies the services that must be made available for the component to execute as intended. The existence of a 'requires' interface does not compromise the independence or deployability of a component, as it only defines *what* services are needed, not *how* those services should be provided.
- **UML Notation:** UML notation is often used to represent component interfaces, with "ball and socket" symbols illustrating how interfaces can fit together.

Component Access Components are typically accessed using Remote Procedure Calls (RPCs). Each component is assigned a unique identifier, often a URL, allowing it to be referenced from any networked computer. This enables components to be called in a manner similar to invoking a procedure or method running on a local computer.

Component Models A component model is a comprehensive definition of standards governing the implementation, documentation, and deployment of components.

- **Examples of Component Models:** Prominent examples include the EJB model (Enterprise Java Beans), the COM+ model (part of .NET), and the CORBA Component Model.
- **Interface Definition:** The component model explicitly specifies how interfaces should be defined, including the elements that must be part of an interface definition.

Basic Elements of a Component Model Component models define several fundamental elements:

- **Interfaces:** Component models dictate how interfaces are defined, including details such as operation names, parameters, and exceptions that should be part of the interface definition.
- **Usage:** For components to be distributed and accessed remotely, the component model requires that they have a globally unique name or handle associated with them.
- **Deployment:** The component model includes specifications on how components should be packaged for deployment.

Middleware Support Component models form the foundation for middleware, which provides the necessary support for executing components. Implementations of component models typically offer two types of services:

- **Platform Services:** These services enable communication between components that are written according to the specific model.
- **Support Services:** These are application-independent services that can be used by various components. To utilize the services provided by a component model, components are deployed within a 'container'. A container is essentially a set of interfaces that grant access to the service implementations.

CBSE Processes

CBSE processes are software processes specifically designed to support component-based software engineering. These processes account for the opportunities for reuse and the distinct activities involved in developing and utilizing reusable components.

Two Main CBSE Processes:

1. **Development for Reuse:** This process focuses on creating components or services that are intended to be reused in multiple applications. It often involves generalizing existing components to make them more broadly applicable.
2. **Development with Reuse:** This process involves developing new applications by integrating and utilizing existing components and services.

Supporting Processes Alongside the two main processes, several supporting processes are crucial for effective CBSE:

- **Component Acquisition:** This is the process of obtaining components, either for direct reuse or for further development into reusable components. Acquisition may

involve sourcing components developed internally or finding them from external providers.

- **Component Management:** This concerns the ongoing management of an organization's reusable components. It involves ensuring that components are properly cataloged, stored, and readily available for reuse. Information about component usage and tracking different versions are also part of this process.
- **Component Certification:** This process involves checking the quality of a component and certifying that it meets specified standards or requirements. Certification usually means that an independent party, separate from the original developer, verifies the component's quality.

CBSE for Reuse (Component Development for Reuse) CBSE for reuse primarily focuses on the development of components themselves.

- **Generalization:** Components initially developed for a specific application typically need to be generalized to enhance their reusability.
- **Stable Domain Abstractions:** A component is most likely to be reusable if it is associated with a stable domain abstraction, often referred to as a business object. For example, in a hospital system, stable domain abstractions would include entities like nurses, patients, or treatments.
- **Component Reusability Characteristics:**
 - Should reflect stable domain abstractions.
 - Should hide its internal state representation.
 - Should be as independent as possible, minimizing external dependencies.
 - Should publish exceptions through its component interface.
- **Trade-off between Reusability and Usability:** There is an inherent trade-off; a more general interface (leading to greater reusability) often results in increased complexity and, consequently, reduced usability.

Changes for Reusability To make components more reusable, specific modifications are often required:

- Removing application-specific methods.
- Changing names to be more general and abstract.
- Adding methods to broaden the component's functional coverage.
- Ensuring consistent exception handling across the component.
- Adding a configuration interface to allow for component adaptation.
- Integrating required components internally to reduce external dependencies.

Exception Handling in Components Components generally should not handle exceptions internally because each application using the component will have its own specific requirements for exception handling. Instead, a component should define which exceptions can arise during its operation and publish these as part of its interface. However, two practical problems exist with this approach:

- **Bloated Interfaces:** Publishing every possible exception can lead to excessively large and complex interfaces, making them harder to understand and potentially deterring potential users.

- **Dependency on Local Handling:** The correct operation of a component might depend on its own internal exception handling mechanisms. Changing these or externalizing them could have serious implications for the component's functionality.

Legacy System Components Existing legacy systems that still perform useful business functions can be re-packaged as components to facilitate their reuse. This process typically involves developing a 'wrapper component' that implements the necessary 'provides' and 'requires' interfaces, which then access the underlying legacy system. While this re-packaging can be costly, it is often significantly less expensive than completely rewriting the entire legacy system.

Reusable Components (Cost and Performance) The development cost for creating reusable components might be higher than for developing specific, one-time use equivalents. This additional cost associated with enhancing reusability should ideally be borne by the organization as a whole, rather than being allocated to a single project. Furthermore, generic reusable components may be less space-efficient and might have longer execution times compared to their highly specific counterparts.

Component Management Component management encompasses several key activities:

- **Classification:** Deciding how to categorize components so they can be easily discovered by potential users.
- **Availability:** Making the component accessible, either by storing it in a central repository or by offering it as a service.
- **Information Maintenance:** Keeping track of information regarding the component's usage.
- **Version Tracking:** Managing and tracking different versions of the component.
- **Certification:** Organizations with a reuse program may perform some form of component certification before a component is made available for reuse. This certification signifies that someone other than the original developer has verified the component's quality.

CBSE with Reuse The CBSE with reuse process is primarily concerned with finding suitable reusable components and integrating them into new systems.

- **Trade-offs:** When reusing components, it is crucial to make trade-offs between the ideal system requirements and the actual services offered by available components.
- **Process Steps:** This process typically involves an iterative approach:
 1. Developing outline requirements for the new system.
 2. Searching for existing components that might meet these requirements.
 3. Modifying the initial requirements based on the functionality actually available from the discovered components.
 4. Conducting another search to determine if there are better components that align with the revised requirements.
 5. Composing the selected components to construct the new system.

Component Identification Issues Several challenges arise during component identification:

- **Trust:** It is essential to be able to trust the supplier of a component. An untrusted component might not function as advertised, or, in the worst case, it could pose a security risk.
- **Requirements:** Different groups of components will satisfy distinct sets of requirements, making the selection process complex.
- **Validation:**
 - Component specifications may lack sufficient detail to allow for the development of comprehensive tests.
 - Components might possess unwanted functionality, and it can be difficult to test whether this unintended functionality will interfere with the target application.

Component Validation Component validation involves:

- Developing a set of test cases for a component. These might be new test cases or extensions of those provided with the component.
- Developing a test harness to execute these component tests. The main challenge in component validation is that the component specification might not be detailed enough to enable the creation of a complete and exhaustive set of tests. Furthermore, in addition to verifying that a reusable component performs the required functions, it is also important to check that it does not contain any malicious code or unwanted functionality.

Ariane Launcher Failure – Validation Failure Case Study

- **Background:** In 1996, the inaugural test flight of the Ariane 5 rocket tragically ended in disaster, with the launcher losing control just 37 seconds after takeoff.
- **Problem Cause:** The failure was attributed to a reused component from a previous version of the launcher, specifically the Inertial Navigation System.
- **Underlying Issue:** Assumptions made during the development of this reused component for the Ariane 4 did not hold true for the Ariane 5.
- **Relevance:** Crucially, the specific functionality within this component that failed was not even required for the Ariane 5 mission. This incident highlights the critical importance of thorough validation of reused components, especially when their operating context changes.

Component Composition

Definition Component composition is the process of assembling individual components to create a complete software system. This involves integrating components with each other and with the underlying component infrastructure. Typically, 'glue code' must be written to facilitate this integration.

Types of Composition There are three primary types of component composition:

1. **Sequential Composition:** In this type, the composed components are executed one after another in a predefined sequence. This involves composing the 'provides' interfaces of each component, where the output of one component serves as the input for the next.

2. **Hierarchical Composition:** Here, one component relies on and calls upon the services provided by another component. This type of composition involves connecting the 'provides' interface of one component with the 'requires' interface of another.
3. **Additive Composition:** This method involves combining the interfaces of two or more components to form a new, larger component. The resulting 'provides' and 'requires' interfaces of the integrated component are a combination of the interfaces of its constituent components.

Glue Code Glue code is the custom code written to enable different components to work together seamlessly.

- **Functionality:** If components A and B are composed sequentially, the glue code would be responsible for calling component A, collecting its results, and then calling component B using those results. It may also transform the results into the specific format required by component B.
- **Resolving Incompatibilities:** Glue code is frequently used to resolve incompatibilities between component interfaces.

Interface Incompatibility Interface incompatibilities can arise in several forms:

- **Parameter Incompatibility:** This occurs when operations have the same name but expect or return parameters of different data types.
- **Operation Incompatibility:** This happens when the names of operations within the interfaces being composed are different, even if they perform similar functions.
- **Operation Incompleteness:** This situation arises when the 'provides' interface of one component is only a subset of the 'requires' interface of another component, meaning the providing component doesn't offer all the services the requiring component needs.

Adaptor Components Adaptor components are specifically designed to address the problem of component incompatibility by reconciling the interfaces of components that need to be composed. Different types of adaptors are required depending on the specific type of composition being performed.

- **Example:** An `addressFinder` component and a `mapper` component might be composed through an adaptor named `postCodeStripper`. This adaptor strips the postal code from an address provided by the `addressFinder` and then passes only the postal code to the `mapper` component, facilitating their sequential composition.

Interface Semantics When composing components, one must rely on the component documentation to determine if interfaces that are syntactically compatible are also semantically compatible.

- **Example: Photo Library Component:** Consider an `addItem` method in a `PhotoLibrary` component. Its documentation states: "This method adds a photograph to the library and associates the photograph identifier and catalogue descriptor with the photograph".
- **Semantic Ambiguities:** Even with this description, semantic questions can arise:
 - What happens if the photograph identifier already exists in the library? Does it overwrite, throw an error, or do nothing?

- Is the photograph descriptor associated with the catalogue entry as well as the photograph itself? For example, if a photograph is deleted, is its catalogue information also deleted? These questions highlight that syntax alone is insufficient for confident composition.

The Object Constraint Language (OCL) The Object Constraint Language (OCL) is specifically designed to define constraints that are associated with UML models. It is based on the concept of pre and post condition specification, a common practice in many formal methods.

OCL Description of the Photo Library Interface OCL can provide precise semantic meaning to interface operations using `pre` (precondition) and `post` (postcondition) statements.

```
-- The context keyword names the component to which the conditions apply
context addItem
-- The preconditions specify what must be true before execution of addItem
pre: PhotoLibrary.libSize() > 0 PhotoLibrary.retrieve(pid) = null

-- The postconditions specify what is true after execution
post: libSize () = libSize()@pre + 1 PhotoLibrary.retrieve(pid) = p
PhotoLibrary.catEntry(pid) = photodesc

context delete
pre: PhotoLibrary.retrieve(pid) <> null ;
post: PhotoLibrary.retrieve(pid) = null PhotoLibrary.catEntry(pid) =
PhotoLibrary.catEntry(pid)@pre PhotoLibrary.libSize() = libSize()@pre-1
```

Photo Library Conditions (Derived from OCL) As specified by the OCL for the Photo Library component:

- **addItem Precondition:** Before adding an item (`addItem`):
 - The library must already contain at least one item (`PhotoLibrary.libSize() > 0`). (Note: The source text states `>0`, but the interpretation usually implies an existing, non-empty library, or simply that the library itself has been initialized and exists before adding a *new* item, possibly `libSize() >= 0` if it's the first item. The source states "The library must exist - assume that creating a library adds a single item to it").
 - There must not be an existing photograph in the library with the same identifier (`pid`) as the photograph to be entered (`PhotoLibrary.retrieve(pid) = null`).
- **addItem Postcondition:** After adding an item (`addItem`):
 - The size of the library increases by one (`libSize() = libSize()@pre + 1`).
 - Retrieving using the same identifier will return the photograph that was just added (`PhotoLibrary.retrieve(pid) = p`).
 - Looking up the catalogue using that identifier will return the catalogue entry that was made (`PhotoLibrary.catEntry(pid) = photodesc`).
- **delete Precondition:** Before deleting an item (`delete`):
 - A photograph with the specified identifier must exist in the library (`PhotoLibrary.retrieve(pid) <> null`).
- **delete Postcondition:** After deleting an item (`delete`):

- Retrieving using the identifier will now return null, indicating the photograph has been removed (`PhotoLibrary.retrieve(pid) = null`).
- The catalogue entry associated with the deleted photo remains unchanged (or is implied to be removed/unlinked) (`PhotoLibrary.catEntry(pid) = PhotoLibrary.catEntry(pid)@pre`).
- The size of the library decreases by one (`PhotoLibrary.libSize() = libSize()@pre-1`).

Composition Trade-offs When composing components, conflicts often arise between different types of requirements:

- **Functional vs. Non-functional Requirements:** There may be conflicts between achieving the desired functional capabilities and meeting non-functional requirements such as performance, security, or reliability.
- **Rapid Delivery vs. System Evolution:** Balancing the need for quick system delivery with the desire for future system evolvability can be challenging.

Decisions that need to be made during composition include:

- Determining which composition of components will effectively deliver the specified functional requirements.
- Considering which composition allows for future changes and adaptability of the system.
- Predicting and understanding the emergent properties that will arise from the composed system.

Key Points

- CBSE is a reuse-based approach for defining and implementing loosely coupled components into systems.
- A software component is a unit whose functionality and dependencies are fully defined by its interfaces.
- Components can be implemented either as executable elements included directly within a system or as external services.
- A component model establishes a set of standards that component providers and composers must adhere to.
- The primary CBSE processes are "CBSE for reuse" (focused on component development) and "CBSE with reuse" (focused on application development using components).
- During the CBSE process, the activities of requirements engineering and system design are often interleaved.
- Component composition is the process of connecting or 'wiring' components together to form a complete system.
- When composing reusable components, it is typically necessary to write adaptor components to reconcile differences in their interfaces.
- When choosing component compositions, developers must consider the required functionality, non-functional requirements, and the long-term system evolution.

Would you like me to review the concept of "Component Interfaces" in more detail, or perhaps we can move on to a short quiz to test your understanding of "CBSE Processes"?